Introduction

Low-Level Design (LLD) interviews often test your ability to design core components that are reusable, configurable, and efficient. A common example used to evaluate these skills is the **Logging Framework**.

In this article, we'll guide you through a structured 9-step approach to design a logging system much like you would in a real interview scenario. At every step, we'll share pro tips to help you clearly communicate your design choices, showcase scalability considerations, and leave a strong impression on the interviewer.

Step 1: Clarify Requirements

Functional Requirements

These contain the core logging operations which are as follows:

1. Log Levels with Priority System:

- Support 5 log levels: DEBUG, INFO, WARNING, ERROR, FATAL
- Each level has a priority (DEBUG=1, INFO=2, WARNING=3, ERROR=4, FATAL=5)
- Only log messages with priority ≥ configured level
- Example: If level is set to WARNING, only WARNING, ERROR, and FATAL messages are logged:
- logger.setLevel(LogLevel.WARNING);
- logger.debug("This won't be logged"); // Skipped
- logger.info("This won't be logged"); // Skipped
- logger.warning("This will be logged"); // Logged

logger.error("This will be logged"); // Logged

2. Log Message Structure:

- Each log message contains: timestamp, level, message text, and optional source
- **Timestamp:** When the log was created
- **Level:** Severity of the message
- **Message:** What happened
- **Source:** Which class/method generated the log (optional)
- // Creates: [2024-01-15 10:30:45] [ERROR] [PaymentService.processPayment] -
- // Payment failed for user 123

logger.error("Payment failed for user {}", userId);

3. Multiple Output Destinations:

- Console: Display logs in terminal/console (for development)
- File: Save logs to a file (for production)
- Database: Store logs in database (for analysis)
- Same log message can go to multiple destinations simultaneously

4. Configuration System:

- Set logging level for entire application
- Choose which output destinations to use
- Configure formatting rules
- Simple configuration without complex filtering

5. Thread Safety:

- Multiple threads can log simultaneously without data corruption
- No lost or mixed-up log messages
- Thread-safe operations for all logging components

6. Extensibility:

- Easy to add new output destinations (email, network, cloud storage)
- Easy to add new log levels if needed
- Easy to add custom formatting

7. Message Formatting:

- Customize how log messages appear in output
- Control timestamp format, level display, and message layout
- Different formats for different destinations

Interview Tip: *Confirm with the interviewer which destinations, formats, and levels are must-have features before designing the architecture. This ensures your solution meets their expectations while showing your attention to detail.*

Non-Functional Requirements

- **Thread Safety:** Handle concurrent logging without data corruption
- **Performance:** Minimal overhead for logging operations
- **Extensibility:** Easy to add new log levels and destinations
- **Configurability:** Runtime configuration changes
- **Memory Efficiency:** Reasonable memory usage

Edge Cases to Consider

- Multiple threads logging simultaneously (writing in same lines)
- Invalid log levels or configurations
- File system full during file logging
- Database connection failure during database logging

Step 2: Identify Core Entities

1. LogLevel

- **Enum:** DEBUG, INFO, WARNING, ERROR, FATAL
- **priority:** int (for comparison)
- **isGreaterOrEqual(LogLevel other):** boolean

2. LogMessage

- **timestamp:** Timestamp
- **level:** LogLevel
- **message:** String
- **source:** String (optional - class/method name)

3. LogConfiguration

A simple configuration for the logging framework.

- **rootLevel:** LogLevel

Interview Tip: *Use enums for fixed sets like log levels to ensure type safety and make comparisons easier. It also improves code readability.*

Step 3: Visualize Interaction Flows

Basic Logging Flow

- Application creates log message
- Logger processes message
- If message passes level check, Logger sends to output destinations
- Each destination writes the message

Configuration Flow (Real-time)

- Application sets **LogConfiguration**

- Logger updates its settings
- All future logs follow new configuration

Multi-threaded Flow

- Multiple threads create log messages simultaneously
- Thread-safe Logger processes each request
- Each destination handles concurrent writes safely

Formatting Flow

- **LogMessage** reaches destination
- Destination formats message
- Formatted message is written to output

Interview Tip: *Visual flow diagrams (sequence diagrams) help interviewers quickly grasp how different components interact. Keep them simple and focused on key steps.*

Step 4: Define Class Structures and Relationships

1. Core Interfaces (Fundamental Classes and Interfaces)

- **Logger**
    - void debug(String message)
    - void info(String message)
    - void warning(String message)
    - void error(String message)
    - void fatal(String message)
    - void log(LogLevel level, String message)
    - void setLevel(LogLevel level)
    - void addAppender(LogAppender appender)
    - void addFilter(LogFilter filter)
    - void removeFilter(LogFilter filter)

    - List<LogAppender> getAppenders()
    - List<LogFilter> getFilters()
- **LogAppender**
    - void append(LogMessage message)
    - void setLevel(LogLevel level)
    - LogLevel getLevel()
    - boolean isEnabled(LogLevel level)
    - void setFormatter(LogFormatter formatter)
    - LogFormatter getFormatter()
- **LogFormatter**
    - String format(LogMessage message)
    - void setPattern(String pattern)
    - String getPattern()
    - void setDateFormat(String dateFormat)
- **LogFilter**
    - boolean shouldLog(LogMessage message)
    - void setLevel(LogLevel level)
    - LogLevel getLevel()
- **LogConfiguration**
    - void setRootLevel(LogLevel level)
    - LogLevel getRootLevel()

---

2. Implementation Classes

- **ConsoleAppender implements LogAppender**
    - Writes to System.out/System.err based on level
    - Uses formatter to format messages before output
- **FileAppender implements LogAppender**
    - Writes to specified file with timestamp
    - Uses formatter to format messages before writing
- **DatabaseAppender implements LogAppender**

    - Writes to database table
    - Uses formatter to format messages before storage
- **SimpleFormatter implements LogFormatter**
    - Default format: "[LEVEL] TIMESTAMP - MESSAGE"
    - Configurable date format and pattern
- **DetailedFormatter implements LogFormatter**
    - Extended format: "[LEVEL] TIMESTAMP [SOURCE] - MESSAGE"
    - Includes source information when available
- **LevelFilter implements LogFilter**
    - Filters messages based on minimum log level
    - Only allows messages with level ≥ configured level
- **SourceFilter implements LogFilter**
    - Filters messages based on source/class name
    - Can include or exclude specific packages/classes

---

3. Core Classes

- **LogLevel**
    - Enum with priority values
    - isGreaterOrEqual(LogLevel other) method
- **LogMessage**
    - Immutable data class
    - Builder pattern for construction

Interview Tip**:** *Grouping interfaces, implementations, and core classes separately helps you explain your design in a structured way, making it easier for interviewers to follow.*

Step 5: Core Use Cases & Methods

Basic Logging Use Case

- Application calls `logger.info("message")`
- `LoggerImpl.log(LogLevel.INFO, "message")` is invoked
- `LogMessage.Builder().level(INFO).message("message").build()` creates the log message
- Check `level.isGreaterOrEqual(loggerLevel)`
- For each appender: `appender.isEnabled(level)`
- `appender.append(logMessage)`
- `appender.getFormatter().format(logMessage)`
- Write formatted message to destination

Configuration Use Case

- Application calls `logger.setLevel(LogLevel.WARNING)`
- `LoggerImpl.setLevel(LogLevel.WARNING)` updates configuration
- Future `logger.log()` calls use new level for filtering

Multi-threaded Use Case

- Thread1: `logger.info("msg1")` + Thread2: `logger.error("msg2")`
- `LoggerImpl.log()` uses `synchronized` keyword for thread safety
- Uses `Collections.synchronizedList` for appenders/filters
- Concurrent `appender.append()` calls are handled safely
- No data corruption occurs

Filtering Use Case

- `LoggerImpl.log()` creates `LogMessage`
- For each filter in filters list: `filter.shouldLog(logMessage)`
- If any filter returns `false` → message is dropped (return early)
- If all filters pass → proceed to appenders

Formatting Use Case

- `appender.append(logMessage)` is called
- `LogFormatter formatter = appender.getFormatter()`
- `String formatted = formatter.format(logMessage)`
- Write formatted string to destination (console, file, database)

Interview Tip: *Explaining use cases step-by-step shows that you understand both the high-level flow and the low-level method calls involved in the system.*

Step 6: OOP Principles & Design Patterns

Design Patterns Used

- **Strategy Pattern** – Used for different appenders (Console, File, Database) and formatters (Simple, Detailed)
- **Chain of Responsibility Pattern** – Implemented in the filter chain processing
- **Builder Pattern** – Used for constructing `LogMessage` objects

OOP Principles Applied

- **Single Responsibility** – Each class has one clear and distinct purpose
- **Open/Closed Principle** – Easy to add new appenders without modifying existing code
- **Liskov Substitution** – All appenders are interchangeable without breaking functionality
- **Interface Segregation** – Clean `LogAppender` interface with only required methods
- **Dependency Inversion** – Depend on `LogAppender` interface, not specific implementations
- **Encapsulation** – Internal state is protected with clear public APIs

---

SOLID Principles

- **Single Responsibility** – Logger handles logging, Appenders handle output, Formatters handle formatting, Filters handle filtering
- **Open/Closed** – New appenders, formatters, and filters can be added without changing existing code
- **Liskov Substitution** – Any `LogAppender`, `LogFormatter`, or `LogFilter` can replace another
- **Interface Segregation** – Each interface only has the methods necessary for its responsibility
- **Dependency Inversion** – Logger depends on interfaces (`LogAppender`, `LogFormatter`, `LogFilter`) instead of concrete classes

---

Step 7: Handle Edge Cases

Edge Case Solutions

- **Multiple Threads Logging:**
    - Use synchronized methods or concurrent collections (e.g., ConcurrentLinkedQueue)
    - Implement thread-safe appender implementations
    - Use atomic operations for shared state (AtomicInteger, AtomicLong)
- **Invalid Log Levels:**
    - Validate inputs in the `LogLevel` enum (parse safely)
    - Default to **ERROR** level for invalid inputs
    - Return clear error messages or throw well-documented exceptions
- **File System Full:**
    - Wrap file operations in try-catch blocks
    - Fallback to console logging or an in-memory buffer
    - Emit alerts or escalate errors to monitoring systems
- **Database Connection Failure:**
    - Use connection pooling and retry logic with backoff
    - Gracefully fallback to file or console logging
    - Persist failed log writes in a retry queue for later flush
- **Invalid Format Patterns:**
    - Validate format patterns in formatter implementations
    - Fallback to a safe/simple format when pattern is invalid
    - Return clear error messages describing pattern syntax issues
- **Filter Configuration Errors:**
    - Validate filter parameters at registration time
    - Default to accept-all behavior for invalid filters (fail-open)
    - Handle filter exceptions gracefully so logging isn't disrupted

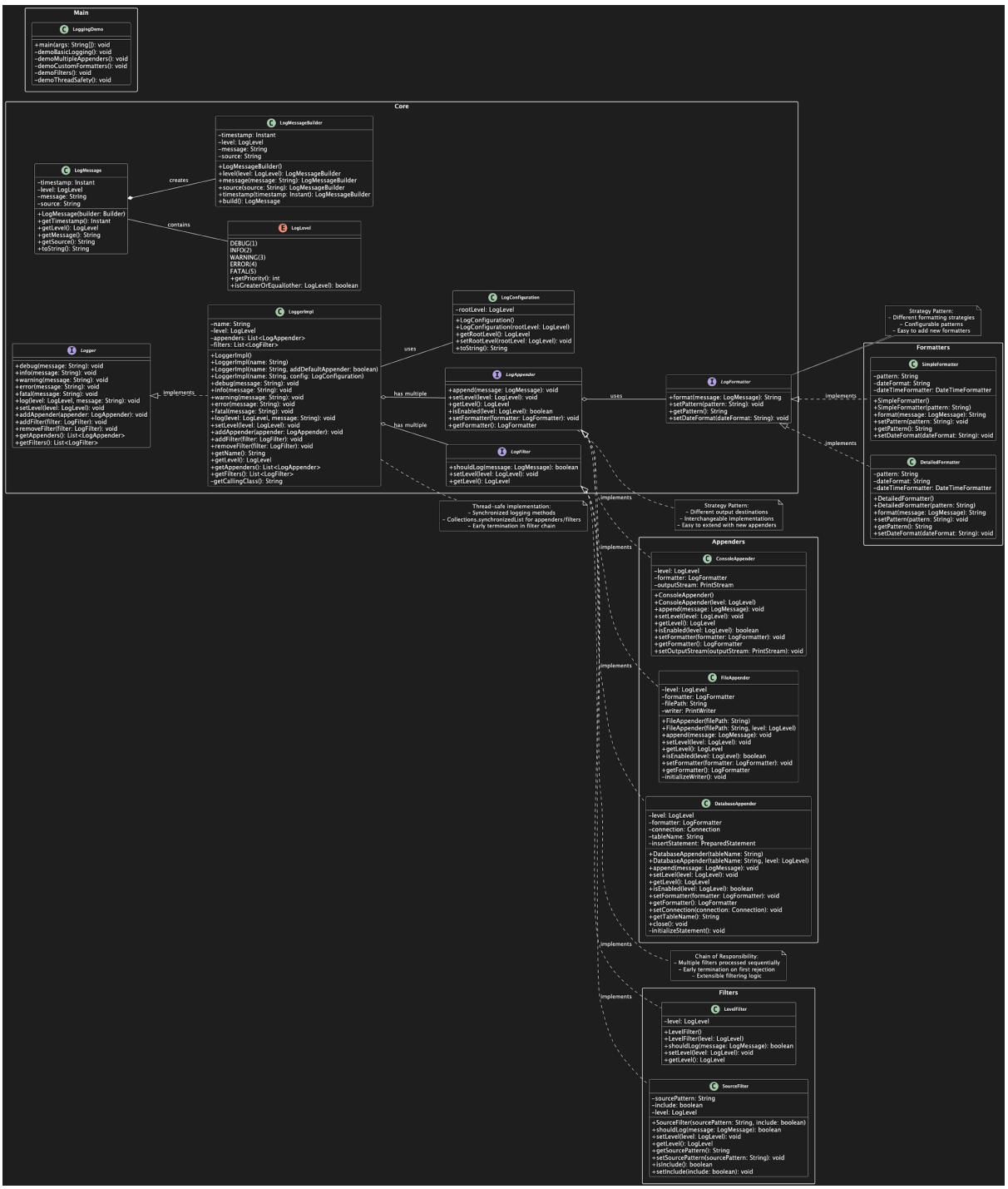
---

Implementation Strategies

- **Thread Safety:** Use synchronized blocks, concurrent collections, or lock-free structures where appropriate.
- **Error Handling:** Wrap critical operations in try-catch and provide fallbacks (console, file, retry-queue).
- **Validation:** Validate all configuration and public API inputs before applying them.
- **Resource Management:** Ensure proper cleanup (close file handles, DB connections) in appenders using try-with-resources or finally blocks.
- **Filter Chain:** Process filters sequentially and stop on first rejection; isolate filter exceptions to avoid breaking logging.
- **Formatting:** Use a template/pattern approach for message formatting and validate patterns up-front.
- **Configuration:** Apply configuration changes atomically (e.g., swap config object references) and validate before swap to avoid inconsistent state.

Interview Tip: *When explaining edge-case handling, state trade-offs (e.g., fail-open vs fail-closed, sync vs async write) and why you chose a particular fallback — interviewers value trade-off awareness as much as the solution itself.*

Class Diagram


Main
LoggingDemo
+main(args: String[]): void
-demoBasicLogging(): void
-demoMultipleAppenders(): void
-demoCustomFormatters(): void
-demoFilters(): void
-demoThreadSafety(): void
Core
LogMessageBuilder
-timestamp: Instant
-level: LogLevel
-message: String
-source: String
+LogMessageBuilder()
+level(level: LogLevel): LogMessageBuilder
+message(message: String): LogMessageBuilder
+source(source: String): LogMessageBuilder
+timestamp(timestamp: Instant): LogMessageBuilder
+build(): LogMessage
LogMessage
-timestamp: Instant
-level: LogLevel
-message: String
-source: String
+LogMessage(builder: Builder)
+getTimestamp(): Instant
+getLevel(): LogLevel
+getMessage(): String
+getSource(): String
+toString(): String
creates
contains
LogLevel
DEBUG(1)
INFO(2)
WARNING(3)
ERROR(4)
FATAL(5)
+getPriority(): int
+isGreaterOrEqual(other: LogLevel): boolean
LogConfiguration
-rootLevel: LogLevel
+LogConfiguration()
+LogConfiguration(rootLevel: LogLevel)
+getRootLevel(): LogLevel
+setRootLevel(rootLevel: LogLevel): void
+toString(): String
uses
LoggerImpl
-name: String
-level: LogLevel
-appenders: List<LogAppender>
-filters: List<LogFilter>
+LoggerImpl()
+LoggerImpl(name: String)
+LoggerImpl(name: String, addDefaultAppender: boolean)
+LoggerImpl(name: String, config: LogConfiguration)
+debug(message: String): void
+info(message: String): void
+warning(message: String): void
+error(message: String): void
+fatal(message: String): void
+log(level: LogLevel, message: String): void
+setLevel(level: LogLevel): void
+addAppender(appender: LogAppender): void
+addFilter(filter: LogFilter): void
+removeFilter(filter: LogFilter): void
+getName(): String
+getLevel(): LogLevel
+getAppenders(): List<LogAppender>
+getFilters(): List<LogFilter>
-getCallingClass(): String
Logger
+debug(message: String): void
+info(message: String): void
+warning(message: String): void
+error(message: String): void
+fatal(message: String): void
+log(level: LogLevel, message: String): void
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
+addAppender(appender: LogAppender): void
+addFilter(filter: LogFilter): void
+removeFilter(filter: LogFilter): void
+getAppenders(): List<LogAppender>
+getFilters(): List<LogFilter>
implements
has multiple
has multiple
LogAppender
+append(message: LogMessage): void
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
+isEnabled(level: LogLevel): boolean
+setFormatter(formatter: LogFormatter): void
+getFormatter(): LogFormatter
LogFilter
+shouldLog(message: LogMessage): boolean
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
uses
LogFormatter
+format(message: LogMessage): String
+setPattern(pattern: String): void
+getPattern(): String
+setDateFormat(dateFormat: String): void
implements
implements
Strategy Pattern:
- Different formatting strategies
- Configurable patterns
- Easy to add new formatters
Formatters
SimpleFormatter
-pattern: String
-dateFormat: String
-dateTimeFormatter: DateTimeFormatter
+SimpleFormatter()
+SimpleFormatter(pattern: String)
+format(message: LogMessage): String
+setPattern(pattern: String): void
+getPattern(): String
+setDateFormat(dateFormat: String): void
DetailedFormatter
-pattern: String
-dateFormat: String
-dateTimeFormatter: DateTimeFormatter
+DetailedFormatter()
+DetailedFormatter(pattern: String)
+format(message: LogMessage): String
+setPattern(pattern: String): void
+getPattern(): String
+setDateFormat(dateFormat: String): void
Thread-safe implementation:
- Synchronized logging methods
- Collections.synchronizedList for appenders/filters
- Early termination in filter chain
implements
Strategy Pattern:
- Different output destinations
- Interchangeable implementations
- Easy to extend with new appenders
Appenders
ConsoleAppender
-level: LogLevel
-formatter: LogFormatter
-outputStream: PrintStream
+ConsoleAppender()
+ConsoleAppender(level: LogLevel)
+append(message: LogMessage): void
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
+isEnabled(level: LogLevel): boolean
+setFormatter(formatter: LogFormatter): void
+getFormatter(): LogFormatter
+setOutputStream(outputStream: PrintStream): void
implements
FileAppender
-level: LogLevel
-formatter: LogFormatter
-filePath: String
-writer: PrintWriter
+FileAppender(filePath: String)
+FileAppender(filePath: String, level: LogLevel)
+append(message: LogMessage): void
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
+isEnabled(level: LogLevel): boolean
+setFormatter(formatter: LogFormatter): void
+getFormatter(): LogFormatter
-initializeWriter(): void
implements
DatabaseAppender
-level: LogLevel
-formatter: LogFormatter
-connection: Connection
-tableName: String
-insertStatement: PreparedStatement
+DatabaseAppender(tableName: String)
+DatabaseAppender(tableName: String, level: LogLevel)
+append(message: LogMessage): void
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
+isEnabled(level: LogLevel): boolean
+setFormatter(formatter: LogFormatter): void
+getFormatter(): LogFormatter
+setConnection(connection: Connection): void
+getTableName(): String
+close(): void
-initializeStatement(): void
implements
Chain of Responsibility:
- Multiple filters processed sequentially
- Early termination on first rejection
- Extensible filtering logic
Filters
LevelFilter
-level: LogLevel
+LevelFilter()
+LevelFilter(level: LogLevel)
+shouldLog(message: LogMessage): boolean
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
implements
SourceFilter
-sourcePattern: String
-include: boolean
-level: LogLevel
+SourceFilter(sourcePattern: String, include: boolean)
+shouldLog(message: LogMessage): boolean
+setLevel(level: LogLevel): void
+getLevel(): LogLevel
+getSourcePattern(): String
+setSourcePattern(sourcePattern: String): void
+isinclude(): boolean
+setInclude(include: boolean): void